

24 | Rust图像识别：利用YOLOv8识别对象

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。这节课我们来学习如何使用 Rust 对图片中的对象进行识别。

图像识别是计算机视觉领域中的重要课题，而计算机视觉又是 AI 的重要组成部分，相当于 AI 的眼睛。目前图像识别领域使用最广泛的框架是 YOLO，现在已经迭代到了 v8 版本。而基于 Rust 机器学习框架 Candle，可以方便地实现 YOLOv8 算法。因此，这节课我们继续使用 Candle 框架来实现图片的识别。

Candle 框架有一个示例，演示了 YOLOv8 的一个简化实现。我在此基础上，将这个源码中的示例独立出来，做成了一个单独的项目，方便你学习（查看 [🔗代码地址](#)）。

注：这节课的代码适用于 candle_core v0.3 版本。

YOLO 简介

YOLO (You Only Look Once) 是一种目标检测算法，它可以在一次前向传递中检测出图像中的所有物体的位置和类别。因为只需要看一次，YOLO 被称为 Region-free 方法，相比于 Region-based 方法，YOLO 不需要提前找到可能存在目标的区域 (Region)。YOLO 在 2016 年被提出，发表在计算机视觉顶会 CVPR (Computer Vision and Pattern Recognition) 上。YOLO 对整个图片进行预测，并且它会一次性输出所有检测到的目标信息，包括类别和位置。

YOLO 也使用神经网络进行图像识别，一般来说，如果是推理的话，我们需要一个神经网络的预训练模型文件。下面你会看到，在运行示例的时候，会自动从 HuggingFace 下载对应的预训练模型。

YOLOv8 的模型结构比起之前的版本，会复杂一些，我们来看一下官方整理的图片。

M: middle, 模型比 small 大。

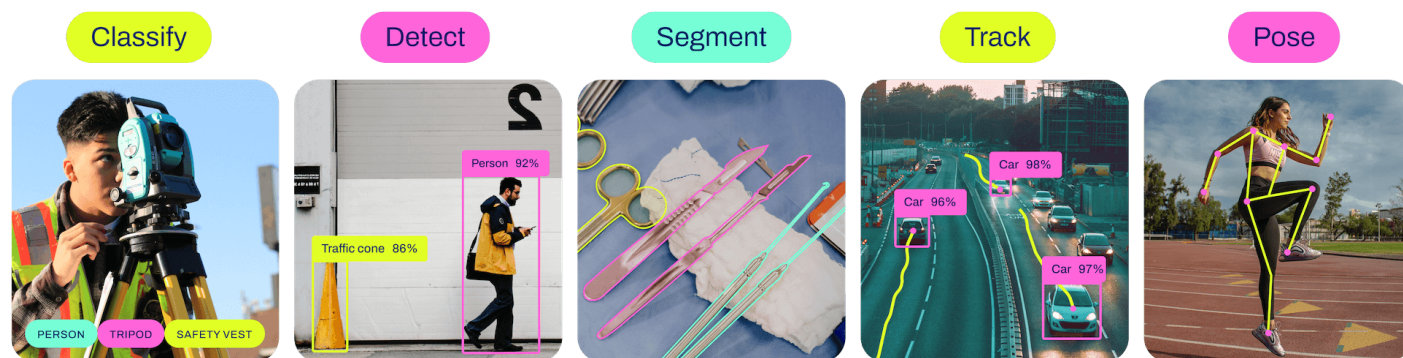
L: large, 模型比 middle 大, 比 x 小。

X: extra large, 模型最大。探测速度最慢, 精度最高。

在下面的示例中, 我们可以通过参数来指定选择哪个模型。

YOLOv8 的能力

YOLO 发展到第 8 代已经很强大了。它可以对图像做分类、探测、分段、轨迹、姿势等。



图片来源: <https://raw.githubusercontent.com/ultralytics/assets/main/im/banner-tasks.png>

了解了 YOLO 的能力, 下面我们开始实际用起来。

动手实验

下载源码:

```
1 git clone https://github.com/miketang84/jikeshijian
2 cd jikeshijian/24-candle_yolov8
```

 复制代码

物体探测

假设我们有这样一张图片。



编译运行下面这行代码。

 复制代码

```
1 cargo run --release -- assets/football.jpg --which m
```

请注意，这个运行过程中，会联网从 HuggingFace 上下载模型文件，需要科学上网环境。

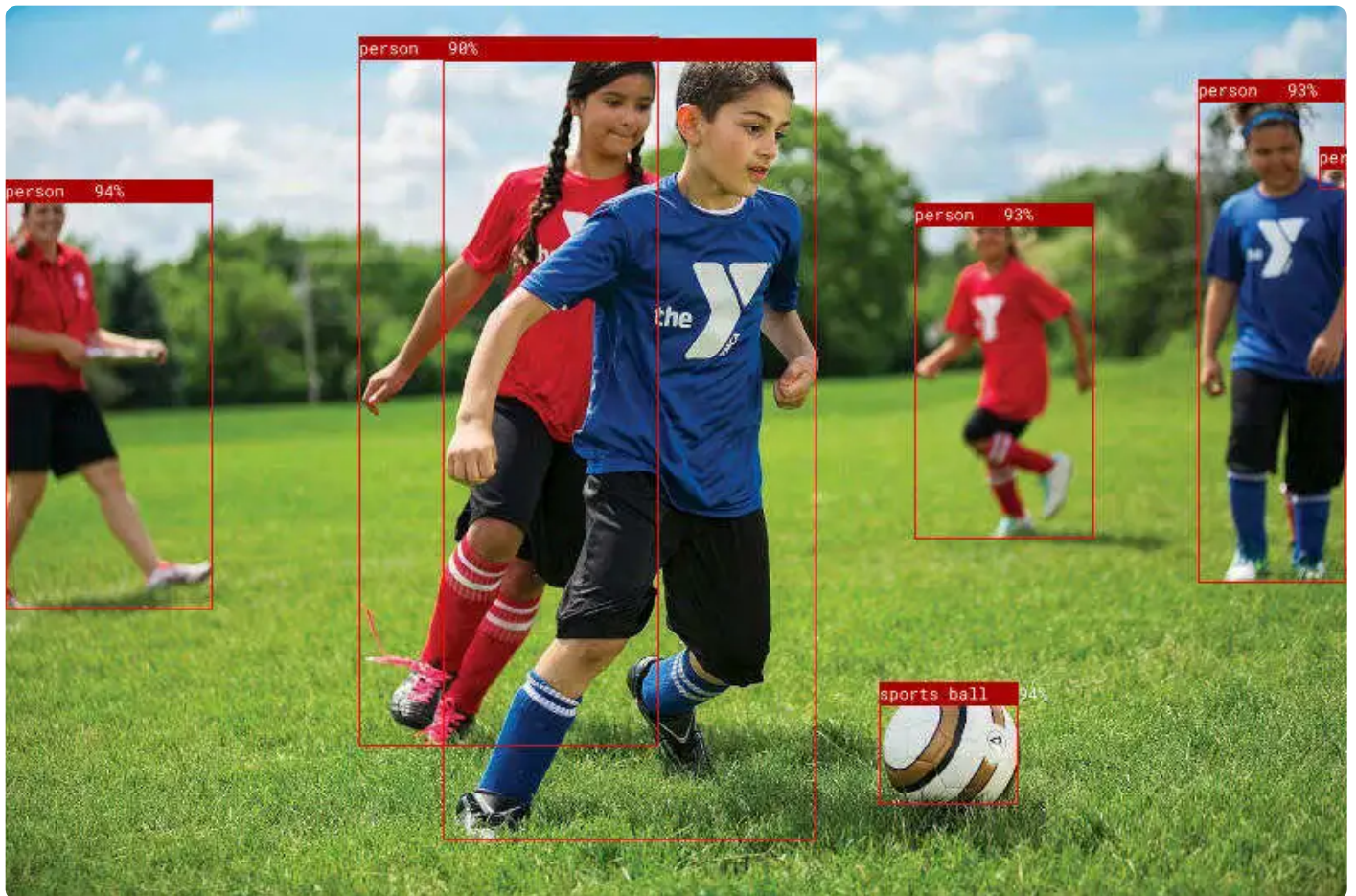
运行输出：

 复制代码

```
1 $ cargo run --release -- assets/football.jpg --which m
2 ProxyChains-3.1 (http://proxychains.sf.net)
3     Finished release [optimized] target(s) in 0.08s
4     Running `target/release/candle_demo_yolov8 assets/football.jpg --which m`
5 Running on CPU, to run on GPU, build this example with `--features cuda`
6 model loaded
7 processing assets/football.jpg
8 generated predictions Tensor[dims 84, 5460; f32]
```

```
9 person: Bbox { xmin: 0.15629578, ymin: 81.735344, xmax: 99.46689, ymax: 281.7202,  
10 person: Bbox { xmin: 433.88196, ymin: 92.59643, xmax: 520.25476, ymax: 248.76715,  
11 person: Bbox { xmin: 569.20465, ymin: 34.737877, xmax: 639.8049, ymax: 269.4999,  
12 person: Bbox { xmin: 209.33649, ymin: 16.313568, xmax: 388.09424, ymax: 388.7763,  
13 person: Bbox { xmin: 169.212, ymin: 15.2717285, xmax: 312.59946, ymax: 345.16046,  
14 person: Bbox { xmin: 626.709, ymin: 65.91608, xmax: 639.791, ymax: 86.72856, conf  
15 sports ball: Bbox { xmin: 417.45734, ymin: 315.16333, xmax: 484.62384, ymax: 372.  
16 writing "assets/football.pp.jpg"
```

在 assets 目录下生成 football.pp.jpg 文件，打开后效果如下：



可以看到，Yolo 正确识别了 6 个人，和一个运动球。

姿势探测

我们来看一下，对同一张图片，运行姿势探测的效果。


```
1 cargo run --release -- assets/football.jpg --which m --task pose
```

我们的工具在 assets 目录下生成 football.pp.jpg 文件，打开后效果如下：



效果是不是很 cool。下面我们详细解释一下这次实战的代码。

源码解释

YOLOv8 神经网络模型的原理比较复杂，这节课我们主要讲解这个示例中 Rust 的用法，从中可以学到不少 Rust 相关知识。

```
1 // #[cfg(feature = "mkl")]
2 // extern crate intel_mkl_src;
3
4 // #[cfg(feature = "accelerate")]
5 // extern crate accelerate_src;
6
```

📄 复制代码

```
7 mod model;
8 use model::{Multiples, YoloV8, YoloV8Pose};
9 mod coco_classes;
10
11 use candle_core::utils::{cuda_is_available, metal_is_available};
12 use candle_core::{DType, Device, IndexOp, Result, Tensor};
13 use candle_nn::{Module, VarBuilder};
14 use candle_transformers::object_detection::{non_maximum_suppression, Bbox, KeyPoi
15 use clap::{Parser, ValueEnum};
16 use image::DynamicImage;
17
18 // Keypoints as reported by ChatGPT :)
19 // Nose
20 // Left Eye
21 // Right Eye
22 // Left Ear
23 // Right Ear
24 // Left Shoulder
25 // Right Shoulder
26 // Left Elbow
27 // Right Elbow
28 // Left Wrist
29 // Right Wrist
30 // Left Hip
31 // Right Hip
32 // Left Knee
33 // Right Knee
34 // Left Ankle
35 // Right Ankle
36 const KP_CONNECTIONS: [(usize, usize); 16] = [
37     (0, 1),
38     (0, 2),
39     (1, 3),
40     (2, 4),
41     (5, 6),
42     (5, 11),
43     (6, 12),
44     (11, 12),
45     (5, 7),
46     (6, 8),
47     (7, 9),
48     (8, 10),
49     (11, 13),
50     (12, 14),
51     (13, 15),
52     (14, 16),
53 ];
54
55
```



```

56
57 // 获取设备, Cpu还是Cuda或Metal
58 pub fn get_device(cpu: bool) -> Result<Device> {
59     if cpu {
60         Ok(Device::Cpu)
61     } else if cuda_is_available() {
62         Ok(Device::new_cuda(0)?)
63     } else if metal_is_available() {
64         Ok(Device::new_metal(0)?)
65     } else {
66         #[cfg(all(target_os = "macos", target_arch = "aarch64"))]
67         {
68             println!(
69                 "Running on CPU, to run on GPU(metal), build this example with `--f
70             );
71         }
72         #[cfg(not(all(target_os = "macos", target_arch = "aarch64")))]
73         {
74             println!("Running on CPU, to run on GPU, build this example with `--f
75         }
76         Ok(Device::Cpu)
77     }
78 }
79 // 报告对象探测的结果, 以及用图像处理工具在图上画出来标注
80 pub fn report_detect(
81     pred: &Tensor,
82     img: DynamicImage,
83     w: usize,
84     h: usize,
85     confidence_threshold: f32,
86     nms_threshold: f32,
87     legend_size: u32,
88 ) -> Result<DynamicImage> {
89     let pred = pred.to_device(&Device::Cpu)?;
90     let (pred_size, npreds) = pred.dims2()?;
91     let nclasses = pred_size - 4;
92
93     let mut bboxes: Vec<Vec<Bbox<Vec<KeyPoint>>>> = (0..nclasses).map(|_| vec![])
94     // 选出符合置信区间的结果
95     for index in 0..npreds {
96         let pred = Vec::<f32>::try_from(pred.i(.., index))??;
97         let confidence = *pred[4..].iter().max_by(|x, y| x.total_cmp(y)).unwrap()
98         if confidence > confidence_threshold {
99             let mut class_index = 0;
100             for i in 0..nclasses {
101                 if pred[4 + i] > pred[4 + class_index] {
102                     class_index = i
103                 }
104             }

```

```

105         if pred[class_index + 4] > 0. {
106             let bbox = Bbox {
107                 xmin: pred[0] - pred[2] / 2.,
108                 ymin: pred[1] - pred[3] / 2.,
109                 xmax: pred[0] + pred[2] / 2.,
110                 ymax: pred[1] + pred[3] / 2.,
111                 confidence,
112                 data: vec![],
113             };
114             bboxes[class_index].push(bbox)
115         }
116     }
117 }
118
119 non_maximum_suppression(&mut bboxes, nms_threshold);
120
121 // 在原图上标注, 并打印标注的框的信息
122 let (initial_h, initial_w) = (img.height(), img.width());
123 let w_ratio = initial_w as f32 / w as f32;
124 let h_ratio = initial_h as f32 / h as f32;
125 let mut img = img.to_rgb8();
126 let font = Vec::from(include_bytes!("roboto-mono-stripped.ttf") as &[u8]);
127 let font = rusttype::Font::try_from_vec(font);
128 for (class_index, bboxes_for_class) in bboxes.iter().enumerate() {
129     for b in bboxes_for_class.iter() {
130         println!("{}", coco_classes::NAMES[class_index], b);
131         let xmin = (b.xmin * w_ratio) as i32;
132         let ymin = (b.ymin * h_ratio) as i32;
133         let dx = (b.xmax - b.xmin) * w_ratio;
134         let dy = (b.ymax - b.ymin) * h_ratio;
135         if dx >= 0. && dy >= 0. {
136             imageproc::drawing::draw_hollow_rect_mut(
137                 &mut img,
138                 imageproc::rect::Rect::at(xmin, ymin).of_size(dx as u32, dy as u32),
139                 image::Rgb([255, 0, 0]),
140             );
141         }
142         if legend_size > 0 {
143             if let Some(font) = font.as_ref() {
144                 imageproc::drawing::draw_filled_rect_mut(
145                     &mut img,
146                     imageproc::rect::Rect::at(xmin, ymin).of_size(dx as u32, dy as u32),
147                     image::Rgb([170, 0, 0]),
148                 );
149                 let legend = format!(
150                     "{} {:.0}% ",
151                     coco_classes::NAMES[class_index],
152                     100. * b.confidence
153                 );

```

```

154         imageproc::drawing::draw_text_mut(
155             &mut img,
156             image::Rgb([255, 255, 255]),
157             xmin,
158             ymin,
159             rusttype::Scale::uniform(legend_size as f32 - 1.),
160             font,
161             &legend,
162         )
163     }
164 }
165 }
166 }
167 Ok(DynamicImage::ImageRgb8(img))
168 }
169 // 报告姿态探测的结果，以及用图像处理工具在图上画出来标注
170 pub fn report_pose(
171     pred: &Tensor,
172     img: DynamicImage,
173     w: usize,
174     h: usize,
175     confidence_threshold: f32,
176     nms_threshold: f32,
177 ) -> Result<DynamicImage> {
178     let pred = pred.to_device(&Device::Cpu)?;
179     let (pred_size, npreds) = pred.dims2()?;
180     if pred_size != 17 * 3 + 4 + 1 {
181         candle_core::bail!("unexpected pred-size {pred_size}");
182     }
183     let mut bboxes = vec![];
184     // 选出符合置信区间的结果
185     for index in 0..npreds {
186         let pred = Vec::<f32>::try_from(pred.i(.., index))??;
187         let confidence = pred[4];
188         if confidence > confidence_threshold {
189             let keypoints = (0..17)
190                 .map(|i| KeyPoint {
191                     x: pred[3 * i + 5],
192                     y: pred[3 * i + 6],
193                     mask: pred[3 * i + 7],
194                 })
195                 .collect::<Vec<_>>();
196             let bbox = Bbox {
197                 xmin: pred[0] - pred[2] / 2.,
198                 ymin: pred[1] - pred[3] / 2.,
199                 xmax: pred[0] + pred[2] / 2.,
200                 ymax: pred[1] + pred[3] / 2.,
201                 confidence,
202                 data: keypoints,

```



```

203         };
204         bboxes.push(bbox)
205     }
206 }
207
208 let mut bboxes = vec![bboxes];
209 non_maximum_suppression(&mut bboxes, nms_threshold);
210 let bboxes = &bboxes[0];
211
212 // 在原图上标注, 并打印标注的框和姿势的信息
213 let (initial_h, initial_w) = (img.height(), img.width());
214 let w_ratio = initial_w as f32 / w as f32;
215 let h_ratio = initial_h as f32 / h as f32;
216 let mut img = img.to_rgb8();
217 for b in bboxes.iter() {
218     println!("{b:?}");
219     let xmin = (b.xmin * w_ratio) as i32;
220     let ymin = (b.ymin * h_ratio) as i32;
221     let dx = (b.xmax - b.xmin) * w_ratio;
222     let dy = (b.ymax - b.ymin) * h_ratio;
223     if dx >= 0. && dy >= 0. {
224         imageproc::drawing::draw_hollow_rect_mut(
225             &mut img,
226             imageproc::rect::Rect::at(xmin, ymin).of_size(dx as u32, dy as u3
227             image::Rgb([255, 0, 0]),
228         );
229     }
230     for kp in b.data.iter() {
231         if kp.mask < 0.6 {
232             continue;
233         }
234         let x = (kp.x * w_ratio) as i32;
235         let y = (kp.y * h_ratio) as i32;
236         imageproc::drawing::draw_filled_circle_mut(
237             &mut img,
238             (x, y),
239             2,
240             image::Rgb([0, 255, 0]),
241         );
242     }
243
244     for &(idx1, idx2) in KP_CONNECTIONS.iter() {
245         let kp1 = &b.data[idx1];
246         let kp2 = &b.data[idx2];
247         if kp1.mask < 0.6 || kp2.mask < 0.6 {
248             continue;
249         }
250         imageproc::drawing::draw_line_segment_mut(
251             &mut img,

```

```
252         (kp1.x * w_ratio, kp1.y * h_ratio),
253         (kp2.x * w_ratio, kp2.y * h_ratio),
254         image::Rgb([255, 255, 0]),
255     );
256 }
257 }
258 Ok(DynamicImage::ImageRgb8(img))
259 }
260 // 选择模型尺寸
261 #[derive(Clone, Copy, ValueEnum, Debug)]
262 enum Which {
263     N,
264     S,
265     M,
266     L,
267     X,
268 }
269 // 对象探测任务还是姿势探测任务
270 #[derive(Clone, Copy, ValueEnum, Debug)]
271 enum YoloTask {
272     Detect,
273     Pose,
274 }
275 // 命令行参数定义, 基于Clap
276 #[derive(Parser, Debug)]
277 #[command(author, version, about, long_about = None)]
278 pub struct Args {
279     /// 是否运行在CPU上面
280     #[arg(long)]
281     cpu: bool,
282
283     /// 是否记录日志
284     #[arg(long)]
285     tracing: bool,
286
287     /// 模型文件路径
288     #[arg(long)]
289     model: Option<String>,
290
291     /// 用哪一个模型
292     #[arg(long, value_enum, default_value_t = Which::S)]
293     which: Which,
294
295     images: Vec<String>,
296
297     /// 模型置信门槛
298     #[arg(long, default_value_t = 0.25)]
299     confidence_threshold: f32,
300 }
```

```

301     /// non-maximum suppression的閾值
302     #[arg(long, default_value_t = 0.45)]
303     nms_threshold: f32,
304
305     /// 要执行的任务
306     #[arg(long, default_value = "detect")]
307     task: YoloTask,
308
309     /// 标注的字体大小
310     #[arg(long, default_value_t = 14)]
311     legend_size: u32,
312 }
313
314 impl Args {
315     fn model(&self) -> anyhow::Result<std::path::PathBuf> {
316         let path = match &self.model {
317             Some(model) => std::path::PathBuf::from(model),
318             None => {
319                 let api = hf_hub::api::sync::Api::new()?;
320                 let api = api.model("lmz/candle-yolo-v8".to_string());
321                 let size = match self.which {
322                     Which::N => "n",
323                     Which::S => "s",
324                     Which::M => "m",
325                     Which::L => "l",
326                     Which::X => "x",
327                 };
328                 let task = match self.task {
329                     YoloTask::Pose => "-pose",
330                     YoloTask::Detect => "",
331                 };
332                 api.get(&format!("yolov8{size}{task}.safetensors"))?
333             }
334         };
335         Ok(path)
336     }
337 }
338
339 pub trait Task: Module + Sized {
340     fn load(vb: VarBuilder, multiples: Multiples) -> Result<Self>;
341     fn report(
342         pred: &Tensor,
343         img: DynamicImage,
344         w: usize,
345         h: usize,
346         confidence_threshold: f32,
347         nms_threshold: f32,
348         legend_size: u32,
349     ) -> Result<DynamicImage>;

```

```

350 }
351 // YoloV8为对象探测的类型载体
352 impl Task for YoloV8 {
353     fn load(vb: VarBuilder, multiples: Multiples) -> Result<Self> {
354         YoloV8::load(vb, multiples, /* num_classes=*/ 80)
355     }
356
357     fn report(
358         pred: &Tensor,
359         img: DynamicImage,
360         w: usize,
361         h: usize,
362         confidence_threshold: f32,
363         nms_threshold: f32,
364         legend_size: u32,
365     ) -> Result<DynamicImage> {
366         report_detect(
367             pred,
368             img,
369             w,
370             h,
371             confidence_threshold,
372             nms_threshold,
373             legend_size,
374         )
375     }
376 }
377 // YoloV8Pose为姿势探测的类型载体
378 impl Task for YoloV8Pose {
379     fn load(vb: VarBuilder, multiples: Multiples) -> Result<Self> {
380         YoloV8Pose::load(vb, multiples, /* num_classes=*/ 1, (17, 3))
381     }
382
383     fn report(
384         pred: &Tensor,
385         img: DynamicImage,
386         w: usize,
387         h: usize,
388         confidence_threshold: f32,
389         nms_threshold: f32,
390         _legend_size: u32,
391     ) -> Result<DynamicImage> {
392         report_pose(pred, img, w, h, confidence_threshold, nms_threshold)
393     }
394 }
395 // 主体运行逻辑
396 pub fn run<T: Task>(args: Args) -> anyhow::Result<()> {
397     let device = get_device(args.cpu)?;
398     // 选择模型尺寸，加载模型权重参数进来

```



```
399 let multiples = match args.which {
400     Which::N => Multiples::n(),
401     Which::S => Multiples::s(),
402     Which::M => Multiples::m(),
403     Which::L => Multiples::l(),
404     Which::X => Multiples::x(),
405 };
406 let model = args.model()?;
407 let vb = unsafe { VarBuilder::from_mmaped_safetensors(&[model], DType::F32, &
408 let model = T::load(vb, multiples)?;
409 println!("model loaded");
410 for image_name in args.images.iter() {
411     println!("processing {image_name}");
412     let mut image_name = std::path::PathBuf::from(image_name);
413     let original_image = image::io::Reader::open(&image_name)?
414         .decode()
415         .map_err(candle_core::Error::wrap)?;
416     let (width, height) = {
417         let w = original_image.width() as usize;
418         let h = original_image.height() as usize;
419         if w < h {
420             let w = w * 640 / h;
421             //
422             (w / 32 * 32, 640)
423         } else {
424             let h = h * 640 / w;
425             (640, h / 32 * 32)
426         }
427     };
428     let image_t = {
429         let img = original_image.resize_exact(
430             width as u32,
431             height as u32,
432             image::imageops::FilterType::CatmullRom,
433         );
434         let data = img.to_rgb8().into_raw();
435         Tensor::from_vec(
436             data,
437             (img.height() as usize, img.width() as usize, 3),
438             &device,
439         )?
440         .permute((2, 0, 1))?
441     };
442     let image_t = (image_t.unsqueeze(0)?.to_dtype(DType::F32)? * (1. / 255.))
443     let predictions = model.forward(&image_t)?.squeeze(0)?;
444     println!("generated predictions {predictions:?}");
445     let image_t = T::report(
446         &predictions,
447         original_image,
```

```

448         width,
449         height,
450         args.confidence_threshold,
451         args.nms_threshold,
452         args.legend_size,
453     )?;
454     image_name.set_extension("pp.jpg");
455     println!("writing {image_name:?}");
456     image_t.save(image_name)?
457 }
458
459 Ok(())
460 }
461 // 程序入口
462 pub fn main() -> anyhow::Result<()> {
463     use tracing_chrome::ChromeLayerBuilder;
464     use tracing_subscriber::prelude::*;
465
466     let args = Args::parse();
467
468     let _guard = if args.tracing {
469         let (chrome_layer, guard) = ChromeLayerBuilder::new().build();
470         tracing_subscriber::registry().with(chrome_layer).init();
471         Some(guard)
472     } else {
473         None
474     };
475
476     match args.task {
477         YoloTask::Detect => run::<YoloV8>(args)?,
478         YoloTask::Pose => run::<YoloV8Pose>(args)?,
479     }
480     Ok(())
481 }

```

我挑选里面一些重要的内容来讲解一下。

第 7~8 行，加载模型模块。YOLOv8 的模型实现都放在这里面，它在 Candle 的平台基础上实现了一个简易版本的 Darknet 神经网络引擎。第 9 行，加载 coco 数据集分类表。YOLOv8 对数据分成 80 种类别。你可以打开 coco_classes.rs 文件查看。

第 11~14 行，引入 Candle 基础组件。第 15 行引用 clap 赋能命令行功能。这个在上一讲中已经讲过了。第 16 行引入 image crate。我们在这个例子里处理图片使用的是 image 和 imageproc 两个 crate。

第 36 ~ 53 行是人体姿势的参数配置 KP_CONNECTIONS。

第 57 ~ 77 行，是在 candle 中获取能使用的设备的函数。可以看到，Linux 和 Windows 下我们可以使用 CUDA，mac 下我们可以使用 Metal。

第 79 ~ 167 行，report_detect 是第一个任务，对象探测的业务代码。第 169 ~ 258 行，report_pose 是第二个任务，姿势探测的业务代码。这两个任务我们等会儿还会再说到。

第 260 ~ 267 行，定义选用哪个模型，分别对应前面讲到的 N、S、M、L、X。第 269 ~ 273 行，定义对象探测和姿势探测两个不同的任务。第 275 ~ 311 行，定义命令行参数对象 Args，你可以关注一下各个字段的默认值。第 313 ~ 336 行，定义 model 函数，实际是加载到模型的正确路径，如果本地没有，就会从 HuggingFace 上下载。

第 338 ~ 349 行，定义 Task trait，它依赖另外两个 trait：Module 和 Sized。Module 来自 [candle_nn crate](#)，表示神经网络中的一个模块，有向前推理 forward 的功能。Sized 来自 [Rust std 标准库](#)，表示被实现的类型是固定尺寸的。

第 351 ~ 375 行，为 YOLOv8 实现 Task trait，YOLOv8 就是我们用于目标探测的任务承载类型。第 377 ~ 393，为 YOLOv8Pose 实现 Task trait，YOLOv8Pose 就是我们用于姿势探测的任务承载类型。

第 395 ~ 459 行是业务内容。第 461 ~ 480 行是 main 函数，里面做了一些日志配置，并且根据任务类型分配到 YOLOv8 或 YOLOv8Pose 两个不同的任务去。

我们看到，这里使用了 `run::<YoloV8>(args)` 这种写法，再对照 run 的函数签名：

 复制代码

```
1 pub fn run<T: Task>(args: Args) -> anyhow::Result<()> {
```

这个函数签名中有一个类型参数 T，被 Task 约束。根据 [第 10 讲](#) 的内容，我们可以说类型 T 具有 Task 的能力。`::<>` 是 turbofish 语法，用来将具体的类型传递进函数的类型参数中。

进入 `run()` 函数中，我们继续看。第 405、406 行，根据指定的不同的模型，将预训练模型的内容加载成 `model` 实例。第 407 行有个 `T::load()` 写法，实际就是 `YOLOv8` 和 `YOLOv8Pose` 上都实现了 `load()` 关联函数，它定义在 `Task trait` 中。

然后第 409 行可以批量对多个图片进行操作，这个需要你在命令行中传参数指定。我们前面的示例只处理一张图片。然后下面第 415 ~ 426 行，是对图片尺寸的规约化处理。因为 `YOLOV8` 只能在 640px x 640px 的图片上进行检测，所以需要在代码中预处理一下。

第 427 ~ 440 行是将处理后的图片加载成 `Tensor` 对象。第 441 ~ 442 行，执行推理预测。第 444 ~ 452 行，调用各自任务的汇报业务。第 453 ~ 455 行，生成处理后的图片，写入磁盘中。

第 444 行出现了 `T::report()`，解释跟前面一样，实际就是 `YOLOv8` 和 `YOLOv8Pose` 上都实现了 `report()` 关联函数，它定义在 `Task trait` 中。然后这个 `T::report()` 会进一步路由到 `report_detect()` 和 `report_pose()` 函数中，各自调用。

在各自的 `report` 函数中，会对上一步 `YOLOv8` 预测的边框值按置信区间进行筛选，然后对图片添加标注，也就是画那些线和框。这样就生成了我们看到的效果图的内存对象。

到这里为止，全部代码就讲解完成了。细节比较生硬，还是图片好玩！

小结

这节课我们使用 `Rust` 实现了 `Yolov8` 算法探测图像中的对象和人物的姿势。从实现过程来说，并不比 `Python` 版本的实现复杂多少。而且从部署上来讲，`Rust` 编译后就一个二进制可执行文件，对于做成一个软件（后面两讲我们会讲如何用 `GUI` 界面）要方便很多。

另一方面，代码中对于函数的返回值，使用了 `anyhow::Result<T>`。上节课我们讲过，使用 `anyhow` 的返回类型能够大大减少我们的心智负担。

这个版本的 `Yolov8` 的算法，是实现在 `Candle` 框架这个平台上的，你可以研究一下 `model.rs` 文件，可以看到，代码量非常少。因为有了 `Candle` 的基础设施，实现一个新的神经网络算法

其实非常简单。

以前，当我们想学习图像识别的时候，我们就得求助于 Python 或 C++。以后你也可以使用 Rust 玩起来了，我以后会持续地输出关于 Rust 在 AI 领域的应用，你可以持续关注，我们一起推进 Rust 在 AI 领域的影响力。

思考题

请你开启 cuda 或 metal 特性尝试一下，使用不同的预训练模型看一下效果差异。另外你还可以换用不同的图片来测试一下各种识别效果。

欢迎你把你实验的结果分享到评论区，也欢迎你把这节课的内容分享给其他朋友，邀他一起学习 Rust，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新 精选



凤梨 🍍

2023-12-30 来自广东

pytorch怎么转safetensors，没工具下载不了外面的模型

作者回复：我代码里面有，用 huggingface-cli 下载,指定镜像. 可以下载的. 转换格式的话可以用这个方案：<https://github.com/ToluClassics/candle-tutorial>



unistart

2023-12-25 来自湖南

老师，我有一个问题就是猫猫那张图执行姿势探测任务时无法正确识别，这是为什么啊？

```
cargo run --release -- assets/cats.jpg --model candle-yolo-v8/yolov8x-pose.safetensors --which x --task pose
```



```
Compiling candle_demo_yolov8 v0.1.0 (E:\Project\rust-jikeshijian\24-candle_yolov8)
Finished release [optimized] target(s) in 12.69s
Running `target\release\candle_demo_yolov8.exe assets/cats.jpg --model candle-yolo-v
8/yolov8x-pose.safetensors --which x --task pose`
Running on CPU, to run on GPU, build this example with `--features cuda`
model loaded
processing "assets/cats.jpg"
generated predictions Tensor[dims 56, 6300; f32]
writing "assets/cats.pp.jpg"
```

作者回复：因为姿势探测好像只支持人。



蕨火

2023-12-20 来自青海

同问，不联网怎么做？

作者回复：训练yolo跟联网关系不大，可以先把预训练模型下载下来，准备好环境，再把网络断掉训练。训练的过程本身不依赖于网络，只是前期资料（初始模型文件，你的数据集，rust app编译好，等等）准备你得准备好。



My dream

2023-12-19 来自四川

用rust怎么训练录像资源啊？

作者回复：这块儿还没有示例，简单的说，pytorch是怎么训练的，在candle中就对应的实现就行了。原理都是神经网络。Rust在AI这块儿还有大量的工作要做，而Rust社区后面会在这块儿投入巨大的精力。欢迎持续关注。



My dream

2023-12-19 来自四川

如果我们的电脑不联网的情况下，用yolo训练图片资源啊？

作者回复: 训练yolo跟联网关系不大, 可以先把预训练模型下载下来, 准备好环境, 再把网络断掉训练。训练的过程本身不依赖于网络, 只是前期资料准备你得准备好。

共 2 条评论 >



My dream

2023-12-19 来自四川

怎么使用yolo训练图片? 请老师请一下

作者回复: 你指的在自己的图片训练集上进行训练吗? 目前网上有很多相关资料: <https://zhuanlan.zhihu.com/p/650002512> https://blog.csdn.net/weixin_45921929/article/details/132450479 https://blog.csdn.net/qq_40716944/article/details/128648001

Candle这边资料现在非常少, 毕竟是最新刚出的东西, 还在飞速发展中。不过, 后面我会在这块儿持续输出相关内容, 包含yolov8的训练。欢迎持续关注。



Geek_e72251

2024-01-05 来自广东

Error: request error: <https://huggingface.co/lmz/candle-yolo-v8/resolve/main/yolov8m.safetensors>: Connection Failed: Connect error: connection timed out

Caused by:

0: <https://huggingface.co/lmz/candle-yolo-v8/resolve/main/yolov8m.safetensors>: Connection Failed: Connect error: connection timed out

1: connection timed out 一直下不来这个文件, 可以提前下载下来然后放到项目里面吗? 浏览器可以正常下载

